IMPLEMENTATION OF A HARDWARE-EFFICIENT PERCEPTRON ON FPGA FOR LOGIC GATE LEARNING

Amira Jomaa Ebrahim, Amna Alhassan Hamad, Aziza Ebrahim Edrees,
Monya Ashraf fadel, Nour-Alhuda Mailoud Mohammed,
Rbah Masoud Saad, Souad Abdulatif Saleh
Omar Al-Mukhtar University, Facility of Engineering, Al-Bayda, Libya
eng20204018@omu.edu.ly , eng20204084@omu.edu.ly , eng20204186@omu.edu.ly , eng20204189@omu.edu.ly , eng20204222@omu.edu.ly , eng20204217@omu.edu.ly

ABSTRACT

This paper presents the design and FPGA implementation of a hardware-efficient Perceptron neural network capable of learning linearly separable binary logic functions (AND, OR, NAND, NOR). The proposed architecture adopts a Software/Hardware co-design strategy. In the software phase, a Python-based algorithm trains the Perceptron model using floating-point operations to extract optimal weights and bias. In the hardware phase, the architecture is translated into synthesized Verilog HDL, employing a Q6.26 fixed-point arithmetic representation to minimize hardware resource utilization and power consumption. By selecting a power-of-two learning rate ($\eta = 0.125 = 2^{-3}$), complex hardware multipliers are completely replaced with arithmetic right-shift operations. Furthermore, the design incorporates a Finite State Machine (FSM) control unit separated from the datapath, with dedicated wait states to ensure timing stability during post-implementation physical delays. Synthesis and timing simulation results confirm 100% classification accuracy, sub-nanosecond execution latency, and significantly lower dynamic power consumption compared to general-purpose CPU execution.

Keywords **-**Field Programmable Gate Array (FPGA), Perceptron, Machine Learning Hardware, Fixed-Point Arithmetic, Multiplier-Free Architecture, Verilog HDL.

1. Introduction

1.1 Motivation

The exponential growth of Artificial Intelligence (AI) and Machine Learning (ML) applications has created a critical demand for energy-efficient and high-speed hardware accelerators [1], [6]. Traditional Von Neumann architectures, such as Central Processing Units (CPUs) and Graphics Processing Units (GPUs), face significant latency and power consumption bottlenecks due to frequent memory transfers when processing matrix operations. Field Programmable Gate Arrays (FPGAs) offer a highly attractive alternative by enabling massive parallel processing, custom bit-precision, and direct hardware execution of neural network computations at the edge [3] [8].

1.2 Problem Description

Implementing artificial neural networks on embedded FPGAs presents strict hardware constraints regarding Logic Array Blocks (LUTs), Flip-Flops (FFs), and Dedicated Digital Signal Processing (DSP) slices [2]. Standard floating-point neural networks require complex, power-hungry floating-point arithmetic units that quickly exhaust FPGA resources. Furthermore, traditional gradient-based learning requires real-time multiplication operations, which severely limit the system clock frequency and increase dynamic power dissipation [8].

1.3 Objectives of the Work

The main objective of this work is to design, implement, and evaluate a hardware-efficient single-layer Perceptron architecture on an FPGA based on classical neural model principles [7]. The specific goals include:

- Developing a hybrid Software/Hardware co-design workflow where model training is performed in Python and hardware execution is realized in Verilog HDL [4].
- Eliminating expensive hardware multipliers by adopting a fixed-point Q6.26 format and a power-of-two learning rate ($\eta = 2^{-3}$) [2].
- Designing a robust structural architecture separating the Datapath Unit from a Finite State Machine (FSM) Control Unit to guarantee timing alignment [4].
- Evaluating resource utilization, processing latency, and power efficiency across various binary logic gate classifications using industrial EDA tools [5].

1.4 Organisation of the Paper

The rest of this paper is organized as follows: Section 2 describes the methodology, including mathematical modelling, software training, fixed-point translation, and Verilog structural design. Section 3 presents the experimental results, timing simulations, resource utilization, and comparison analysis. Section 4 concludes the paper and highlights directions for future work.

2. Methodology

2.1 Mathematical Model and Software Training

The single-layer Perceptron computes a weighted sum of inputs plus a bias term, followed by a step activation function to classify binary patterns [1], [7]. The weighted summation is expressed as:

$$z = \sum_{i=1}^{n} w_i \; x_i \; + b \tag{1}$$

The output y is determined by the step activation function:

$$y = \begin{pmatrix} 1 & if \; z \geq 0 \\ 0 & if \; z < 0 \end{pmatrix} \tag{2}$$

During the offline software phase implemented in Python, the Perceptron updates weights w_i and bias b iteratively using the error gradient rule [1]:

$$e = d - y \tag{3}$$
$$w_i(t+1) = w_i(t) + \Delta w_i \tag{4}$$
$$b(t+1) = b(t) + \eta \cdot (d - y) \tag{5}$$

where d is the target binary output and η is the learning rate parameter

2.2 Fixed-Point Quantization and Multiplier-Free Design

To avoid the overhead of 32-bit floating-point units on FPGA, a Q6.26 fixed-point format (1 sign bit, 5 integer bits, and 26 fractional bits) is utilized [2]. This precision provides sufficient dynamic range while preserving numerical accuracy during iterative accumulation. To eliminate hardware multipliers, the learning rate is fixed to η=0.125=2^{-3}. Consequently, multiplying the error term by the learning rate is simplified into a 3-bit arithmetic right-shift operator (e $\gg$ 3) [2]. This conversion removes the need for DSP hardware blocks, allowing the complete architecture to be synthesized using basic Look-Up Tables (LUTs) and registers.

2.3 Structural Hardware Architecture (FSM & Datapath)

The proposed FPGA architecture is structured into two main components following standard digital logic design principles [4]:

- **Datapath Unit**: Contains the Q6.26 adders, arithmetic shifters, accumulator registers, and step activation logic.
- **Control Unit (FSM)**: Controls state transitions during execution (IDLE, FETCH, LOAD, SETUP, SUM , SHIFT,WAIT_SUM,CHECK, UPDATE, NEXT).

A dedicated WAIT_SUM state was explicitly introduced into the FSM controller to handle propagation delays across the multi-bit fixed-point adder tree during timing simulations, preventing setup-and-hold time violations before latching the final decision into the output registers.

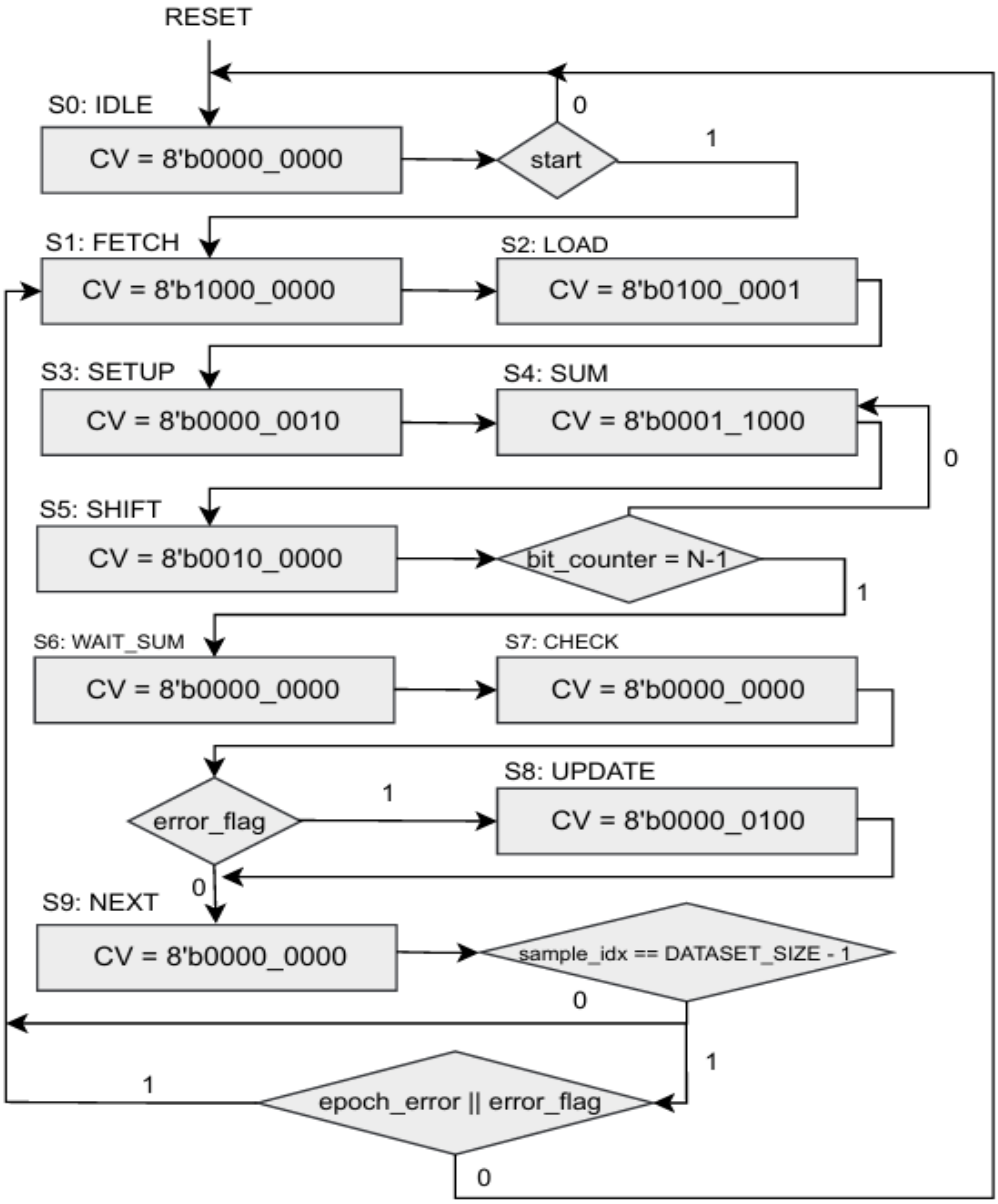


Figure 1
Algorithmic State Machine (ASM) chart describing the sequential execution flow and state transitions of the hardware Perceptron trainer .

Table 1
RTL-CS Table for Structural Perceptron Trainer

State	RTL Code	Asserted Control Signals	CV[7:0]
S0: IDLE	(start ∧ ¬done) / state ← FETCH	None	00000000
S1: FETCH	{vector_x,target} ← M[sample_idx]	enabl_lut	10000000
S2: LOAD	shift_reg ← vector_x, x_hold ← vector_x, target_reg ← target, sum_reg ← 0	enable_sample, sum_reg_reset	01000001
S3: SETUP	sum_reg ← bias_reg	load_bias_to_sum	00000010
S4: SUM	(shift_reg[0]) / sum_reg ← sum_reg + weight_reg[bit_counter]	enable_sum, select_x	00011000
S5: SHIFT	shift_reg ← shift_reg >> 1	enabl_shift	00100000
S6: WAIT_SUM	No RTL operation	None	00000000
S7: CHECK	(sum_reg ≥ 0) → error_flag	None	00000000
S8: UPDATE	(target_reg) / {bias_reg,weight_reg[i]} ← {bias_reg,weight_reg[i]} + LR (¬target_reg) / {bias_reg,weight_reg[i]} ← {bias_reg,weight_reg[i]} - LR	select_bw	00000100
S9: NEXT	(sample_idx < DATASET_SIZE-1) / sample_idx ← sample_idx + 1 (sample_idx = DATASET_SIZE-1 ∧ ¬epoch_error ∧ ¬error_flag) / done ← 1 (sample_idx = DATASET_SIZE-1 ∧ (epoch_error ∨ error_flag)) / epoch ← epoch + 1, sample_idx ← 0	None	00000000

Control vectors (CV) is concatenation of :

enabl_lut & enable_sample & enabl_shift & enable_sum & select_x & select_bw & load_bias_to_sum & sum_reg_reset.

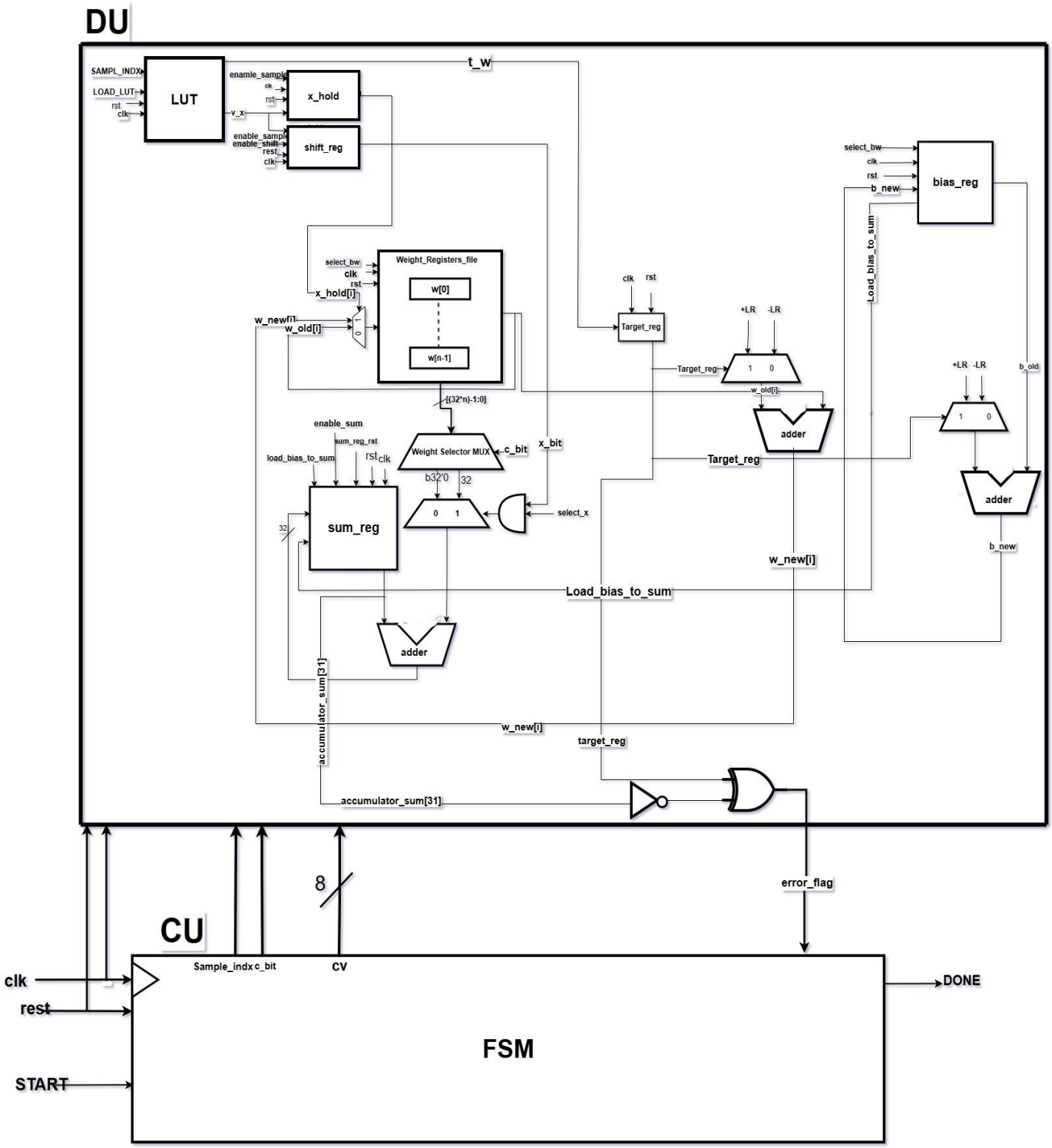


Figure2

illustrates the structural block diagram of the proposed Perceptron architecture, highlighting the separation between the datapath and the FSM controller, along with the multiplier-free shift-based accumulator.

3. *Results*

3.1 Learning Convergence and Accuracy

The proposed Perceptron architecture was evaluated on linearly separable 6-input logic functions (such as 6-input AND, OR, NAND, and NOR gates). The offline Python training script achieved successful convergence for all logic functions, with the required number of epochs varying depending on the specific gate implementation. Table I summarizes the extracted fixed-point parameters, training epochs per gate, and final accuracy achieved after hardware synthesis.

Table 2

Perceptron Parameter Mapping and Hardware Accuracy

LOGIC GATE	Target weights $[w_5, w_4, w_3, w_2, w_1, w_0]$	Target bias	Accuracy
AND	**[0.75, 0.625, 0.5, 0.25,0.125, 0.125]**	**-2.375**	**100%**
OR	**[0.125, 0.125, 0.125, 0.125 ,0.125, 0.125]**	**-0.125**	**100%**
NAND	**[-0.75, -0.625, -0.5, -0.25, -0.125, -0.125]**	**2.25**	**100%**
NOR	**[-0.125, -0.125, -0.125, -0.125, -0.125, -0.125]**	**0**	**100%**

3.2 Resource Utilization and Power Analysis

Post-synthesis resource utilization reports generated via Vivado Design Suite [5] confirm that the multiplier-free Q6.26 design consumes minimal FPGA resources. Because DSP blocks were completely bypassed, the design achieves an exceptionally small footprint.

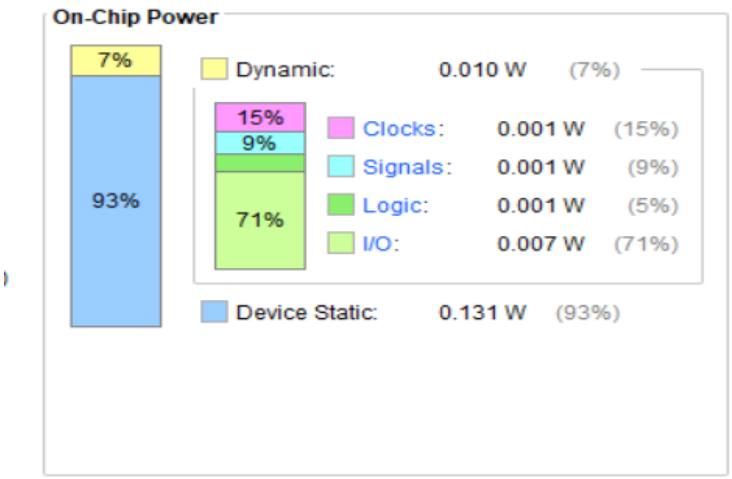
Table 3

Resource Utilization of the Structural Perceptron Architecture inputs=6.

Resource Element	used	available
LUTs	141	133800
Flip-Flops	135	267600
DSPs	0	--
I/O	283	400
BUFGs	1	32

Figure3

Power of the Structural Perceptron Architecture inputs=6.



3.3Hardware vs. Software Performance Comparison

To comparing the FPGA hardware implementation with CPU software execution, the hardware system was evaluated at an operating frequency of 100 MHz. The results demonstrate the clear superiority of dedicated hardware in reducing processing latency and minimizing power consumption compared to traditional software execution.

Table 4

Execution Time (ms) and Performance Comparison between Software and hardware environments (6-inputs)

Environment	AND_GATE	OR_GATE	NAND_GATE	NOR_GATE	Average time
Software	87.000	10.000	71.000	8.000	**44.000**
Hardware	0.6467038	0.0926038	0.6466938	0.0925938	**0.3696488**

4. CONCLUSION

4.1 Review of the Project

This paper successfully demonstrated the full-stream design and FPGA realization of an energy-efficient Perceptron for logic gate classification. By combining Python-based offline training with a Verilog-based fixed-point hardware implementation, the project achieved complete functional accuracy with minimal hardware overhead. The introduction of a multiplier-free shift-based update rule and an explicit FSM wait state ensured both low resource consumption and timing stability.

4.2 Suggestions for Future Work

Future extensions of this research include:

- Scaling the single-layer architecture to Multi-Layer Perceptron to solve non-linearly separable functions (e.g., XOR) [1], [6].
- Implementing on-chip backpropagation learning directly in hardware [8].
- Exploring lower-precision fixed-point representations (e.g., Q4.12) to further optimize memory and power consumption in Edge AI applications [2]

REFERENCES

[1] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2009.

[2] M. A. Ruqaiya and M. S. Jamil, "Fixed-point implementation of artificial neural networks on FPGA," in *Proc. IEEE Int. Conf. Embedded Systems (ICES)*, 2019, pp. 45–50.

[3] V. P. Nambiar, A. A. S., and R. B., "A survey of FPGA-based accelerators for neural networks," *IEEE Access*, vol. 9, pp. 10112–10138, 2021.

[4] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed. New York, NY, USA: McGraw-Hill, 2014.

[5] AMD Xilinx, "Vivado Design Suite User Guide: Synthesis and Implementation," UG901 (v2022.2), Oct. 2022.

[6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.

[7] F. B. Rosenblatt, "The Perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.

[8] A. Shawahna, S. M. Sait, and A. El-Maleh, "FPGA-based accelerators of deep learning networks for learning and classification: A review," *IEEE Access*, vol. 7, pp. 7823–7859, 2019.